Smart Transport Management System

Database Design & OTP-Based Role Access Architecture

---

1. Overview

This document explains the database structure and authentication architecture for the Smart Transport Management System.

The system supports:

- **OTP-based mobile login – (Twillio) support all country**
- **Role-Based Access Control (RBAC)**
- **Module-wise structured permissions**
- **Separate profile data for each role**
- **Secure and scalable access management**

---

2. User Roles in the System

The system supports the following roles:

1. **Buy/Sell – Users who buy or sell goods and request transport**
2. **Shipper – Posts shipment details and manages load requests**
3. **Agent – Coordinates between shipper and transporter**
4. **Transporter – Truck owner/driver who accepts and delivers loads**

---

3. Database Collections Structure (MongoDB)

3.1 Users Collection

This is the main authentication collection. All users log in using mobile number and OTP.

Fields:

- **_id (ObjectId)**
- **name (String)**
- **phone (String – unique)**

- **role (String – buy_sell / shipper / agent / transporter)**
- **isVerified (Boolean)**
- **isActive (Boolean)**
- **createdAt (Date)**
- **updatedAt (Date)**

Example:


```
{
  "_id": "ObjectId",
  "name": "Ravi Kumar",
  "phone": "9876543210",
  "role": "shipper",
  "isVerified": true,
  "isActive": true
}
```

3.2 OTP Codes Collection

Used for OTP-based login validation.

Fields:

- **phone (String)**
- **otp (String – 6 digits)**
- **expiresAt (Date)**
- **isUsed (Boolean)**
- **createdAt (Date)**

Example:


```
{
  "phone": "9876543210",
  "otp": "456789",
```

"expiresAt": "2026-02-13T10:30:00",

"isUsed": false

}

3.3 Modules Collection

Stores all available system modules.

Example Modules:

- **dashboard**
- **load_management**
- **truck_management**
- **user_management**
- **reports**
- **payments**

Example Document:

{

"name": "load_management"

}

3.4 Role Permissions Collection (Module-Based RBAC)

This collection maps roles to structured module-level permissions.

Instead of flat permissions like:

load_create

load_edit

We use module-based structured access control.

Buy/Sell Role

{

```
 "role": "buy_sell",
 "modules": {
  "dashboard": { "access": true },
  "load_management": {
   "create": true,
   "edit": false,
   "delete": false,
   "view": true
  },
  "truck_management": { "view": false, },
  "reports": { "view": false }
 }
}
```

Shipper Role

```
{
 "role": "shipper",
 "modules": {
  "dashboard": { "access": true },
  "load_management": {
   "create": true,
   "edit": true,
   "delete": true,
   "view": true
  },
  "truck_management": {
   "view": true,
```

```
    "create": false,
    "edit": false,
    "delete": false
  },
  "reports": { "view": true }
 }
}
```

Agent Role by sell missing

```
{
 "role": "agent",
 "modules": {
  "dashboard": { "access": true },
  "load_management": { "view": true ,create:true},
  "truck_management": { "view": true },
  "reports": { "view": true }
 }
}
```

Transporter Role

```
{
 "role": "transporter",
 "modules": {
  "dashboard": { "access": true },
  "load_management": { "view": true },
  "truck_management": { "view": true },
  "reports": { "view": false }
```

}

}

---

4. Role-Based Profile Collections

Each role has its own profile collection to keep the database clean and scalable.

4.1 Buy/Sell Profiles

- **userId (Reference to users._id)**
- **companyName**
- **businessType**
- **gstNumber**
- **address**

4.2 Shipper Profiles

- **userId**
- **warehouseLocation**
- **companyName**
- **licenseNumber**

4.3 Agent Profiles

- **userId**
- **agencyName**
- **commissionPercentage**
- **region**

4.4 Transporter Profiles

- **userId**

- **truckNumber**
- **truckType**
- **capacity**
- **currentLocation**
- **availabilityStatus**

---

5. OTP-Based Login Flow

Step 1: User enters mobile number
Step 2: System generates 6-digit OTP
Step 3: OTP stored in otp_codes collection
Step 4: OTP sent via SMS
Step 5: User enters OTP
Step 6: Backend verifies:

- **OTP matches**
- **Not expired**
- **Not already used**
  Step 7: Mark isUsed = true
  Step 8: Fetch user role
  Step 9: Fetch role permissions (modules object)
  Step 10: Generate JWT token

---

6. Login Response Format

After successful OTP verification:


```
{
  "token": "jwt_token_here",
  "user": {
    "name": "Ravi Kumar",
    "role": "shipper",
    "modules": {
      "dashboard": { "access": true },
      "load_management": {
```

```
      "create": true,
      "edit": true,
      "delete": true,
      "view": true
    },
    "truck_management": {
      "view": true
    }
  }
 }
}
```

---

7. Permission Validation (Backend Example)

Before creating a load:

```
if (!user.modules.load_management.create) {
  return res.status(403).json({ message: "Access Denied" });
}
```

Before accessing dashboard:

```
if (!user.modules.dashboard.access) {
  return res.status(403).json({ message: "No Dashboard Access" });
}
```

This ensures secure module-wise access control.

---

8. Architecture Benefits

- **Centralized authentication using mobile OTP**
- **Structured module-based RBAC implementation**
- **Fine-grained permission control (create, edit, delete, view)**

- **Clean separation of role-based profile data**
- **Scalable and extendable architecture**
- **Secure backend-level permission enforcement**

---

9. Conclusion

The Smart Transport Management System uses OTP-based authentication and module-based role access control to ensure secure, scalable, and modular access management.

Each user logs in using a mobile number, and system access is dynamically controlled based on their assigned role and structured module permissions.

10. Truck Management Collection

This collection stores all truck details registered by **transporters**.

Collection: trucks

Fields:

- _id (ObjectId)
- transporterId (Reference → users._id)
- truckNumber (String — unique)
- truckType (String — open / container / trailer / tanker)
- capacity (Number — in tons)
- bodyLength (Number)
- driverName (String)
- driverPhone (String)
- currentLocation (String / GeoJSON)
- availabilityStatus (Boolean)
- rcDocument (String — file URL)
- insuranceDocument (String — file URL)
- fitnessExpiryDate (Date)
- createdAt (Date)
- updatedAt (Date)

```
{
  "truckNumber": "TN01AB1234",
  "transporterId": "ObjectId",
  "truckType": "container",
  "capacity": 20,
  "driverName": "Suresh",
  "driverPhone": "9876543210",
  "currentLocation": "Chennai",
  "availabilityStatus": true
}
```


11. Load Management Collection

This is the **main business collection** where shippers create loads and transporters accept them.

Collection: loads

Fields:

- _id
- loadId (String — unique readable ID like LOAD1001)
- shipperId (Reference → users._id)
- assignedTransporterId (Reference → users._id)
- assignedTruckId (Reference → trucks._id)
- materialType (String)
- weight (Number — tons)
- pickupLocation (String)
- dropLocation (String)
- pickupDate (Date)
- expectedDeliveryDate (Date)
- price (Number)
- status (String)

Status Values:

- posted
- accepted
- in_transit
- delivered
- cancelled
- agentId (Optional Reference → users._id)
- notes (String)
- createdAt
- updatedAt


```
{
  "loadId": "LOAD1001",
  "shipperId": "ObjectId",
  "materialType": "Steel",
  "weight": 15,
  "pickupLocation": "Chennai",
  "dropLocation": "Bangalore",
  "price": 25000,
  "status": "posted"
}
```


12. Load Tracking Collection (Optional but Powerful)

Used to track truck movement during delivery.

Collection: load_tracking

Fields:

- loadId
- truckId
- latitude
- longitude

- locationName
- timestamp

13. Payment / Transaction Collection

Tracks payments between shipper and transporter.

Collection: payments

Fields:

- _id
- loadId
- shipperId
- transporterId
- amount
- paymentMethod (cash / online / wallet / bank)
- paymentStatus (pending / paid / failed)
- transactionId
- paymentDate
- createdAt


```
{
  "loadId": "ObjectId",
  "amount": 25000,
  "paymentMethod": "online",
  "paymentStatus": "paid"
}
```


14. Reports Collection

Reports are usually generated dynamically, but you can store logs for analytics.

Collection: reports

Fields:

- _id
- reportType

Types:

- load_report
- revenue_report
- transporter_performance
- agent_commission
- user_activity
- generatedBy (userId)
- filters (JSON)
- fileUrl (Excel/PDF)
- createdAt


```
{
  "reportType": "revenue_report",
  "generatedBy": "ObjectId",
  "fileUrl": "/reports/revenue_feb.xlsx"
}
```


15. System Activity Logs (Recommended for Production)

Collection: activity_logs

Fields:

- userId
- action
- module
- description
- ipAddress
- createdAt


```
{
  "userId": "ObjectId",
  "action": "CREATE_LOAD",
```

"module": "load_management",

"description": "New load created"

}

16. Complete System Flow (End-to-End)

Load Booking Flow

1. Shipper logs in via OTP
2. Creates load
3. Load status = posted
4. Transporter views available loads
5. Transporter accepts load
6. Truck assigned
7. Status → in_transit
8. Delivery completed
9. Status → delivered
10. Payment processed
11. Reports updated